

📄 App Platform — Deep Dive & Visual Guide

A reusable Flutter monorepo that provides clean, scalable infrastructure so every new project starts with a strong foundation instead of boilerplate.

📄 What Problem Does It Solve?

Every Flutter app needs the same plumbing:

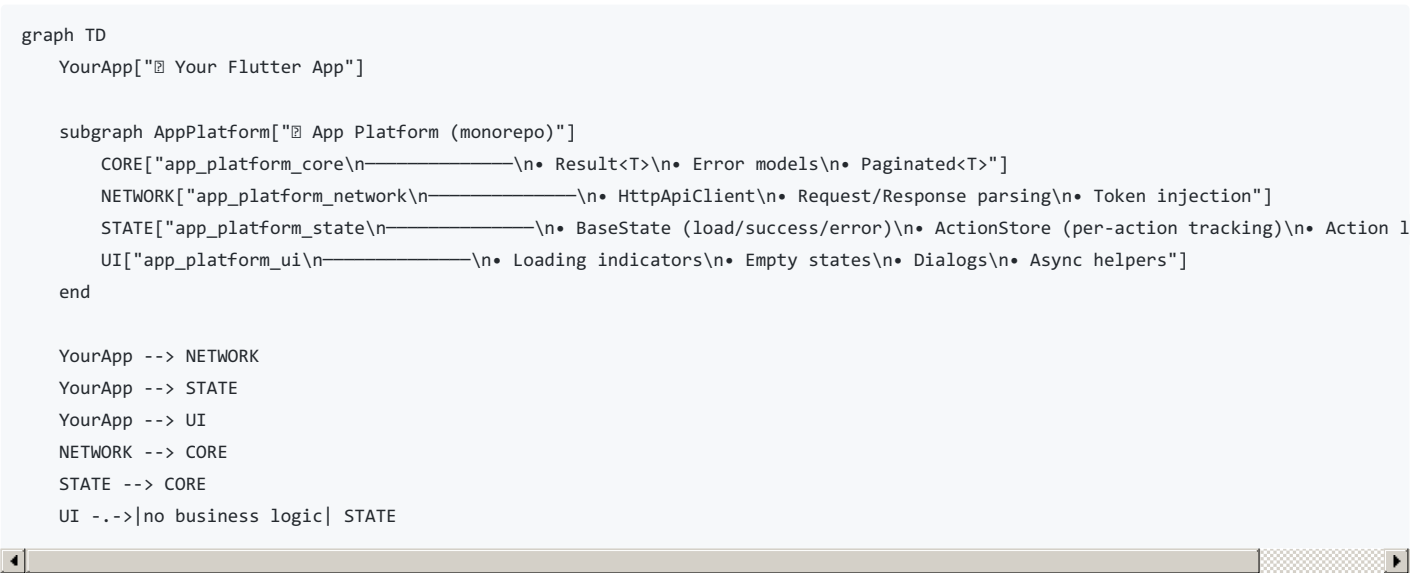
Problem	Without App Platform	With App Platform
API calls	Write HTTP logic from scratch per project	<code>app_platform_network</code> handles it
Loading/error states	Ad-hoc flags everywhere	<code>app_platform_state</code> <code>BaseState</code>
Async result handling	Try/catch spread across codebase	<code>Result<T></code> (Success / Failure)
CRUD actions (create, delete...)	Tangled in screen state	<code>ActionStore</code> tracks each action independently
Pagination	Reinvent every time	<code>Paginated<T></code> model built-in
UI feedback (snackbars, dialogs)	Tightly coupled to business logic	Action listeners decouple UI from logic

📄 Repository Structure

```
app_platform/      ← monorepo root
├─ packages/
│  ├── core/        ← app_platform_core
│  ├── network/     ← app_platform_network
│  ├── state/       ← app_platform_state
│  └─ ui/           ← app_platform_ui
```

Each package is independent and versioned via a **Git commit hash (ref)** — all packages in a consumer project must use the same `ref` for consistency.

📄 Architecture Overview



Dependency rule: `core` is the base. `network` and `state` depend on it. `ui` depends on nothing — it is purely presentational.

📦 Package-by-Package Breakdown

1 📦 app_platform_core

Role: The shared foundation — types, contracts, and models that all other packages use.

Result<T> — Explicit Async Handling

Instead of throwing exceptions (which are invisible at the type level), every async operation returns a `Result<T>`:

```
final result = await repository.getUsers();

switch (result) {
  case Success(:final data):
    print(data);           // 📦 typed data
  case Failure(:final error):
    print(error.message);  // 📦 structured error, never a raw exception
}
```

```
flowchart LR
    AsyncOp["Async Operation\n(API / DB call)"]
    AsyncOp -->|ok| S["Success<T>\n.data → your model"]
    AsyncOp -->|fail| F["Failure<T>\n.error → AppError"]
    S --> Consumer
    F --> Consumer
```

Paginated<T> — Pagination Model

Property	Purpose
<code>items</code>	Current list of items
<code>hasNext</code>	Whether more pages exist
<code>isLoadingMore</code>	Is next page fetching?
<code>paginationError</code>	Error from the last page load

2 📦 app_platform_network

Role: A standardized HTTP layer. Networking logic lives here, completely out of widgets.

HttpClient Setup

```
final apiClient = HttpClient(
  baseUrl: 'https://dummyjson.com',
  client: http.Client(),
  tokenProvider: AppTokenProvider(), // injects auth token automatically
);
```

Making a Typed API Call

```
final result = await apiClient.get<List<User>>(
  '/users',
  parser: (json) =>
    (json['users'] as List).map(User.fromJson).toList(),
);
```

```
sequenceDiagram
    participant Widget
    participant Repository
    participant HttpApiClient
    participant API

    Widget->>Repository: fetchUsers()
    Repository->>HttpApiClient: get('/users', parser: ...)
    HttpApiClient->>API: HTTP GET + Bearer token
    API-->>HttpApiClient: JSON response
    HttpApiClient-->>Repository: Result<List<User>>
    Repository-->>Widget: Result<List<User>>
```

Key design: Parser functions are passed by the caller. `HttpApiClient` handles transport; parsing is explicit per-call. No magic.

3 app_platform_state

Role: State management for screens and discrete user actions — cleanly separated.

It splits state into two concerns:

```
graph LR
    subgraph ScreenState ["BaseState (Screen State)"]
        LS[loading]
        SS[success + data]
        ES[error + message]
    end

    subgraph Actions ["ActionStore (Action State)"]
        A1["ActionKey(create)"]
        A2["ActionKey(delete, id: 5)"]
        A3["ActionKey(update, id: 12)"]
    end
```

BaseState — Screen Loading Cycle

```
switch (state.status) {
    case LoadStatus.loading:
        return const CircularProgressIndicator();
    case LoadStatus.success:
        return UsersList(state.data!);
    case LoadStatus.error:
        return Text(state.error!.message);
}
```

ActionStore — Per-Action Tracking

Each action (delete, update, etc.) has its own lifecycle — no interference with screen data:

```
final key = ActionKey(ActionType.delete, id: user.id).value;

state.start(key);    // marks action as in-progress
// ... await API call ...
state.success(key);  // marks action as done
// or
state.failure(key, error); // marks action as failed
```

```
stateDiagram-v2
    [*] --> idle
    idle --> inProgress : start(key)
    inProgress --> success : success(key)
    inProgress --> failed : failure(key, error)
    success --> [*]
    failed --> [*]
```

UI Feedback via Action Listeners

SnackBars or dialogs listen to action completion **without tight coupling**:

```
Action completes → listener fires → Snackbar/Dialog/Navigate
```

This keeps business logic clean and UI reactive.

4 app_platform_ui

Role: Pure, reusable UI building blocks. Zero business logic.

Widget/Helper	Purpose
Loading indicator	Consistent spinner across app
Empty state widget	"No data" screens
Dialogs	Standardized dialogs
Async UI helpers	Conditional rendering helpers

How to Add to Your Flutter Project

```
# pubspec.yaml
dependencies:
  app_platform_core:
    git:
      url: https://github.com/hassanMohammedDEV/app_platform.git
      ref: <commit-hash>      # ← same hash for all packages!
      path: packages/core

  app_platform_state:
    git:
      url: https://github.com/hassanMohammedDEV/app_platform.git
      ref: <commit-hash>
      path: packages/state

  app_platform_network:
    git:
      url: https://github.com/hassanMohammedDEV/app_platform.git
      ref: <commit-hash>
      path: packages/network

  app_platform_ui:
    git:
      url: https://github.com/hassanMohammedDEV/app_platform.git
      ref: <commit-hash>
      path: packages/ui
```

Then run:

```
flutter pub get
```

[!IMPORTANT] All packages **must use the same** `ref` (commit hash). Mixing versions will cause type incompatibility since `core` types are shared across packages.

🔗 End-to-End Data Flow

```
flowchart TD
    U["👤 User Action\n(tap button)"]
    VM["ViewModel / Controller"]
    REPO["Repository"]
    NET["HttpClient\nnapp_platform_network"]
    API["🌐 Remote API"]
    RESULT["Result<T>\n\napp_platform_core"]
    STATE["BaseState / ActionStore\napp_platform_state"]
    UI_W["Widget Tree\napp_platform_ui"]
    SNACK["SnackBar / Dialog\n(via action listener)"]

    U -->|calls method| VM
    VM -->|delegates| REPO
    REPO -->|HTTP request| NET
    NET -->|GET/POST/...| API
    API -->|JSON| NET
    NET -->|Result<T>| REPO
    REPO -->|Result<T>| VM
    VM -->|updates| STATE
    STATE -->|rebuild| UI_W
    STATE -->|listener fires| SNACK
```

🔗 Design Principles

Principle	How It's Applied
Separation of concerns	Each package has exactly one job
Explicit async handling	<code>Result<T></code> — no hidden exceptions
Predictable error management	Errors are typed values, not thrown objects
Minimal boilerplate	One platform setup, reused across all projects
No global event systems	Clear data flow, no event buses
Long-term maintainability	Packages are pinned by commit hash

🔗 When to Use App Platform

Use Case	Suitable?
Medium–large Flutter apps	🔗 Ideal
Apps with multiple CRUD features	🔗 Ideal
Paginated lists / infinite scroll	🔗 Ideal

Long-term maintained projects Use Case	☒ Ideal Suitable?
Very small prototype / single screen	⚠ May be overkill

Summary: App Platform is a well-structured Flutter monorepo that eliminates repeated boilerplate by centralizing networking, state, error handling, pagination, and UI into four focused, composable packages — letting you focus on product features, not infrastructure.

☒ App Platform — The Master Guide

Deep Dive & Visual Guide Build production-grade Flutter apps with zero boilerplate and maximum scalability.

☒ The Mission

App Platform is more than a collection of packages; it is a **design system for logic**. It standardizes how your apps talk to the internet, handle errors, manage screen states, and respond to user actions.

Why use it?

- **Consistency:** Every feature in every project follows the same pattern.
- **Type Safety:** Errors are values (`Result<T>`), not exceptions.
- **Decoupling:** UI, State, and Networking are strictly separated.
- **Speed:** Start a new project with infrastructure that's already tested and ready.

☒ Visual Architecture

The platform is organized as a monorepo. Each package has a single, focused responsibility.

```
graph TD
    YourApp["☒ Your Flutter App"]

    subgraph AppPlatform["☒ App Platform Ecosystem"]
        direction TB
        CORE["☒ Core\n—————\nTypes, Results, Errors"]
        NET["♥ Network\n—————\nHTTP, Auth, Parsers"]
        STATE["< State\n—————\nBaseState, Actions, Forms"]
        UI["☒ UI\n—————\nAsyncView, Base Widgets"]
    end

    YourApp --> NET
    YourApp --> STATE
    YourApp --> UI
    NET --> CORE
    STATE --> CORE
    UI -.→|Presentational| STATE
```

☒ Deep Dive: `app_platform_core`

The "Brain" of the platform. It defines the language all other packages speak.

1. `Result<T>` — Error Handling as a Value

We never "throw" errors in repositories or services. We return them.

```
// Define an operation
Future<Result<List<User>>> getUsers() async {
  try {
    final data = await api.fetch();
    return Result.success(data);
  } catch (e) {
    return Result.failure(NetworkError(e.toString()));
  }
}

// Consume it
final result = await getUsers();

result.when(
  success: (users) => renderList(users),
  failure: (error) => showError(error.message),
);
```

2. `Paginated<T>` — Infinite Scroll Data Model

Standardized pagination state to keep lists consistent.

Property	Description
<code>items</code>	The accumulated list of data.
<code>hasNext</code>	Boolean indicating if more data exists on the server.
<code>isLoadingMore</code>	Prevents duplicate requests during scroll.

♥ Deep Dive: `app_platform_network`

The "Heart" of the platform. Handles the heavy lifting of HTTP communication.

`HttpApiClient` — The Transport Layer

A wrapper around `http` that handles base URLs, headers, and token injection.

```
sequenceDiagram
    participant App as Flutter App
    participant Client as HttpApiClient
    participant Token as TokenProvider
    participant API as Remote API

    App->>Client: get('/profile')
    Client->>Token: getAccessToken()
    Token-->>Client: "Bearer xyz..."
    Client->>API: HTTP GET + Header
    API-->>Client: JSON Response
    Client->>App: Result<T> (Parsed)
```

Implementation Tip: Always provide a `parser` function. This keeps your API client agnostic of your models.

```
final result = await apiClient.get<User>(
  '/me',
  parser: (json) => User.fromJson(json),
);
```

🔪 Deep Dive: `app_platform_state`

The "Nervous System". It manages how your app reacts to data and user input.

1. `BaseState<T>` — Screen State Management

Every screen usually exists in one of three states: **Loading**, **Success**, or **Error**.

```
// In your Notifier
void load() async {
  state = state.toLoading();
  final result = await repo.getData();
  state = result.toBaseState(); // Automatically maps Success/Failure to BaseState
}
```

2. `ActionStore` — Tracking Discrete Actions

Don't mix screen loading with button actions (like deleting a user). Use the `ActionStore`.

```
stateDiagram-v2
    [*] --> Idle
    Idle --> InProgress : action.start(key)
    InProgress --> Success : action.success(key)
    InProgress --> Failure : action.failure(key, error)
    Success --> Idle : clear
    Failure --> Idle : clear
```

3. `FormNotifier` — Complex Form Logic

Handles validation (Sync & Async), field tracking, and "Touched" states automatically.

```
enum LoginFormFields { email, password }

class LoginNotifier extends FormNotifier<LoginFormFields> {
  LoginNotifier() : super(
    FormStateModel.initial([LoginFormFields.email, LoginFormFields.password]),
    validators: {
      LoginFormFields.email: (val) => val.isEmpty ? 'Required' : null,
    },
  );
}
```

🔪 Deep Dive: `app_platform_ui`

The "Skin". Reusable widgets that understand the Platform's state models.

`AsyncView<T>` — The State Switcher

Instead of writing 100 `if` statements for loading/error, use `AsyncView`.

```
AsyncView<List<User>>>(
  status: state.status,
  data: state.data,
  error: state.error,
  onLoading: () => const ShimmerList(),
  onError: (error) => ErrorWidget(error.message),
  onEmpty: () => const Text("No users found"),
  onSuccess: (users) => ListView(children: users.map(...)),
)
```

🔗 Master Class: Building a Feature

Let's build a "Product List" with a "Delete" action.

Step 1: The Repository

```
class ProductRepository {
  final HttpClient _api;

  Future<Result<List<Product>>> fetch() => _api.get('/products', parser: ...);
  Future<Result<void>> delete(int id) => _api.delete('/products/$id');
}
```

Step 2: The Notifier

```
class ProductNotifier extends StateNotifier<BaseState<List<Product>>> with ActionMixin {
  void load() async {
    state = state.toLoading();
    final result = await _repo.fetch();
    state = result.toBaseState();
  }

  void deleteProduct(int id) async {
    final key = 'delete_$id';
    startAction(key);

    final result = await _repo.delete(id);

    if (result.isSuccess) {
      successAction(key);
      load(); // Refresh list
    } else {
      failureAction(key, result.error!);
    }
  }
}
```

Step 3: The UI

```
Widget build(BuildContext context) {
  return AsyncView(
    status: state.status,
    data: state.data,
    onSuccess: (products) => ListView.builder(
      itemBuilder: (ctx, i) => ListTile(
        title: Text(products[i].name),
        trailing: IconButton(
          actionKey: 'delete_${products[i].id}',
          onPressed: () => ref.read(provider.notifier).deleteProduct(products[i].id),
          child: Icon(Icons.delete),
        ),
      ),
    ),
  );
}
```

⚙️ Setup & Maintenance

1. Installation

Add the packages to your `pubspec.yaml` via Git.

[!IMPORTANT] **The Golden Rule:** All packages MUST use the same `ref` (commit hash).

```
dependencies:
  app_platform_core:
    git:
      url: https://github.com/hassanMohammedDEV/app_platform.git
      ref: <hash>
      path: packages/core
  app_platform_state:
    git:
      url: https://github.com/hassanMohammedDEV/app_platform.git
      ref: <hash>
      path: packages/state
```

2. Versioning Strategy

When you make changes to `app_platform`:

1. Commit and push to the `app_platform` repo.
2. Copy the new commit hash.
3. Update the `ref` in your consumer apps.
4. Run `flutter pub get`.

Summary: App Platform turns infrastructure into a solved problem. By using `Result` for logic, `BaseState` for screens, `ActionStore` for buttons, and `AsyncView` for UI, you create apps that are robust, testable, and beautiful.